Towards Type Systems That Reinterpret Themselves

ANONYMOUS AUTHOR(S)

We advocate that type systems should be able to reinterpret their own terms in any internal model of themselves: equipped with a single tool of reinterpretation, you can express a variety of features, such as universe polymorphism, external parametricity, ability to do synthetic mathematics internally and more. To this end, we explore limits of self-reinterpretation and present two type systems capable of it, namely HM-R and MLTT-R, prove their basic properties and show several examples of useful internal models.

1 Introduction

Consider the following well-known function which maps over elements of the list, written in Agda:

```
map : ∀ {A B : Set} → (A → B) → List A → List B
map f [] = []
map f (x ∷ xs) = f x ∷ map f xs
```

Enjoying both composability of this function and simplicity of its type, we would like to reuse its definition as much as possible. Up to a certain level, we would surely be able to do so: after all, `map` is polymorphic in source and target types, so we can instantiate it with whatever types we want. However, were we to find a magic lamp with a type-theoretical djinn inside, we would ask him to grant us three wishes:

(1) Reuse `map` at higher universe levels, e.g., given a *type operator*, map over *list of types*:

$$(\mathtt{Set} \to \mathtt{Set}) \to \mathtt{List}\,\mathtt{Set} \to \mathtt{List}\,\mathtt{Set}$$

(2) Automagically obtain a proof of the free theorem for `map` – that is, some term of type

$$\begin{aligned}\forall A\,A'\,(Q : A \to A' \to \mathtt{Prop}), \forall B\,B'\,(R : B \to B' \to \mathtt{Prop}),\\ \forall f\,f', (\forall x\,x', Q\,x\,x' \to R(f\,x)\,(f'\,x')) \to\\ \forall (\mathtt{xs} : \mathtt{List}\,A)(\mathtt{xs}' : \mathtt{List}\,A'), Q^*\,\mathtt{xs}\,\mathtt{xs}' \to R^*(\mathtt{map}\,f\,\mathtt{xs})(\mathtt{map}\,f'\,\mathtt{xs}');\end{aligned}$$

(3) Given groupoids A and B and a functor $f : A \to B$, construct a functor $\mathtt{map}\ f$ between groupoids $\mathtt{List}\,A$ and $\mathtt{List}\,B$.

The first wish is, of course, granted by adjusting the definition and making it level-polymorphic. While the theorem in the second wish already holds in systems with parametricity, obtaining a proof *inside* the system without human input is only possible in type systems where parametricity is internalized [1, 3]. The last wish currently has to be fulfilled manually (or with some metaprogramming involved [15]), albeit with a couple of list inductions: some to define a groupoid instance for `List` and few more to prove functoriality of `map`.

We suggest a different approach. Before tweaking the definition of `map`, let's first remember all of the metatheoretic power our type system already has. Abovementioned parametricity of polymorphic definitions is the first result that comes to mind; note that it is obtained by specifying a *relational interpretation* of the typing judgement [16]. And indeed, ones of the most interesting metatheoretic results can be expressed by constructing a certain *model* of a type system: synthetic results in homotopy theory are based on $((\infty, 1)$-)groupoid model of (homotopy) type theory [2, 8]; McBride's usage of displacement for universe polymorphism [13, 14] can be viewed as constructing a model of a type system in higher universes of itself and then interpreting displaced constructions in it. Then, all of our wishes listed above could be granted by constructing a suitable model of a type system *inside* itself (hence we call these *internal models*) and then instantiating `map` in it. Throughout the paper, we'll call type systems with self-reinterpretation capabilities *reflective*.

1.1 Contributions

- Our main goal is to construct an implementable reflective dependently typed system. Main difficulty in doing so arises from a certain no-go result discussed in section 2 which forces us to limit the power of reinterpretation in order to keep type checking decidable.
- For ease of presentation, we first offer a reflective type system based on Hindley-Milner type system called HM-R (section 3). We prove that HM-R is sound, conservative with respect to simply-typed lambda calculus and that global type inference is decidable (section 3.1).
- Building upon the insights gathered from developing HM-R, we're finally equipped to dive into reflective version of Martin-Löf type theory dubbed MLTT-R (section 4). We prove soundness and conservativity of MLTT-R as well (section 4.1).
- As intended, by virtue of reinterpretation, MLTT-R incorporates both a restricted form of universe polymorphism and external parametricity. And, of course, you can use MLTT-R as a system for synthetic mathematics with internal reuse of synthetic results (section 4.2).

2 No-go theorem of internal interpretation

So we want to build a reflective type system. That is, a system where a user is able to both define model of a system internally and apply it to well-typed terms to get their internal interpretations. Naturally, we would like user-defined models to respect definitional equalities of a system; in a dependently typed system, this could be naturally achieved by using an identity type to spell out corresponding equalities inside a model. Then, it would be nice if, before reinterpreting a term, system would first bring it to normal form to get rid of the artifacts of the construction in the result of the interpretation. Finally, in a dependently typed system, it is natural to expect reductions of open terms, so we would like our interpretation to compute on open terms.

The theorem proven below asserts that we cannot have all three in a single system.

First, a couple of definitions. We're going to work inside a classical type-theoretical framework which introduces a sort of contexts Γ, terms t, t' and types T linked by the typing relation $\Gamma \vdash t : T$; computation is specified by one-step reduction relation $t \rightsquigarrow t'$ on terms.

Definition 2.1 (Naïve computational reinterpretation). We call a type operator $F[_]$ a naïve computational reinterpretation of types if it respects computation on types (that is, if there is a reduction relation on types ($\rightsquigarrow$) and $T \rightsquigarrow T'$, then $F[T] \rightsquigarrow^* F[T']$), and computes as a primitive recursive function on normal forms of types. Analogously, term with a hole $[\![_]\!]$ is a naïve computational reinterpretation of terms if it respects computation on terms, computes as a primitive recursive function on normal forms of terms and, if $\Gamma \vdash t : T$, then $\Gamma \vdash [\![t]\!] : F[T]$.

Theorem 2.2 (No-go theorem of naive reinterpretation). *In a confluent type system with a type of functions and naïve computational reinterpretation, if reduction commutes with substitution, for arbitrary well-typed s and t there exists u such that:*

$$\mathsf{app}[\![\lambda x.t]\!][\![s]\!] \rightsquigarrow^* u \ {}^*\!\!\leftsquigarrow [\![t[x \coloneqq s]]\!]$$

where app *is how $[\![_]\!]$ acts on applications.*

Therefore, $\mathsf{app}[\![\lambda x.t]\!][\![s]\!]$ and $[\![t[x \coloneqq s]]\!]$ are definitionally equivalent, which forces beta-reductions in a model to hold definitionally rather than propositionally, directly contradicting our desired outcome. Analogous theorems can be proven for any other reduction present in the system; in other words, **propositional equality is not strong enough**. The proof is by direct example:

Proof. We are given well-typed $\Gamma \vdash s : S$ and $\Gamma, x : S \vdash t : T$. Consider the following term:

$$\Gamma, x : S \to T, y : S \vdash [\![x\, y]\!] : F[T]$$

Term under interpretation is in normal form, hence the whole term computes to $\mathsf{app}[\![x]\!][\![y]\!]$. Substituting $\lambda x.t$ for x and s for y, we get $\mathsf{app}[\![\lambda x.t]\!][\![s]\!]$.

On the other hand, if we perform substitution first, we would get $\Gamma \vdash [\![(\lambda x.t)s]\!] : F[T]$. Since naïve reinterpretation respects reduction, this reduces to $[\![t[x \coloneqq s]]\!]$.

Since reduction commutes with substitution and system is confluent, there should be such u that both terms reduce to. □

Fundamentally, the proof above exploits openness of the original term; if we restrict reinterpretation to compute on closed terms only, we can arrive at a sound system without requiring definitional equalities in models, as is exemplified by our non-dependently-typed system HM-R described in section 3. For dependently typed system, however, our no-go theorem leaves us with three options:

(1) Restrict models to those where equalities hold definitionally rather than propositionally.
(2) Do not proceed with reinterpretation unless term is closed. To allow reasoning on open terms, equations describing behavior of interpretation are exposed as propositional axioms.
(3) Do not reduce open terms under reinterpretation at all, performing reinterpretation as-is. To simplify result of reinterpretation, one can rewrite it with equalities present in the model.

If you view existing work on displacement [13, 14], internal parametricity [1] and displayed type theory [11] through the lens of reinterpretation inside a fixed model (or family of models), you'll see that corresponding models indeed validate equalities definitionally, so it seems that the first approach needs to be taken. It is also the most satisfying from computational point of view. However, if you want a sound system where models can be user-defined, you need to constrain models through typing discipline. By following techniques currently known to us [4], constraining models in such a way would require to define a type of models as a type of algebras of an inductive-inductive-recursive type (or even inductive-inductive-recursive simplicial type, for univalent type theories) which is well beyond the scope of this paper.

Out of two remaining options, the last one is better suited for practical dependently-typed systems since it properly computes on open terms. In addition, it doesn't restrict the class of possible models while at the same time providing the benefits of the first approach: if model is such that equations hold definitionally, no rewriting on the result of reinterpretation is needed to reduce it. The only downside is that, possibly, reductions *after* interpretation are more costly than the ones we could perform *before* it.

3 Almost simply typed almost reflective language: HM-R

We start with a simpler language to iron out the details and actually figure out how to properly reinterpret terms internally before moving on to dependently typed system. The simplest language for which we know a lot of interesting models is simply typed lambda calculus with products, to which we refer to as "STLC" throughout this paper; in this section, we'll present a language capable of interpreting STLC terms as constructions in STLC models which can be defined in our language.

So, what are the models of STLC? In addition to usual set-theoretic semantics where $S \to T$ is interpreted as a function from S to T and $S \times T$ is a cartesian product, there's proof-theoretic semantics that relate terms of type T to intuitionistic proofs of proposition $[\![T]\!]$, where $(\to)$ is now understood to mean implication and $(\times)$ is conjunction. This, of course, is a classic Curry-Howard correspondence [10]; both set-theoretic and proof-theoretic semantics are generalized by a complete description of STLC models as given by Joachim Lambek [12]. According to this *Curry-Howard-Lambek* correspondence, models of STLC are precisely cartesian closed categories (or CCCs for short). Definitions vary; we're going to use the definition 3.1.

Definition 3.1 (Cartesian closed category). Cartesian closed category is the following datum:

(1) A set of *objects* Ob and a family of *morphisms* $\mathrm{Hom}(S,T)$ indexed by $S,T \in \mathrm{Ob}$, where $f \in \mathrm{Hom}(S,T)$ is denoted by $f : S \to T$. In addition, S is called *source* of f, while T is *target* of f, while both S and T together are referred to as *ends* of f.

(2) For each $A \in \mathrm{Ob}$ there's an *identity morphism* $\mathrm{id}_A : A \to A$; for each $f : A \to B$ and $g : B \to C$, there's their *composition* $g \circ f : A \to C$. If a source-target of an identity morphism is clear from context, the subscript is elided. Also, the following properties hold:

$$\mathrm{id} \circ f = f = f \circ \mathrm{id} \qquad\qquad (f \circ g) \circ h = f \circ (g \circ h)$$

for arbitrary morphisms f, g, h with matching ends.

(3) There's a *terminal object* $\top \in \mathrm{Ob}$ such that for each $A \in \mathrm{Ob}$ there's a **unique** morphism $!_A : A \to \top$.

(4) For each $A, B \in \mathrm{Ob}$ there's their *product* $A \times B$ together with *projections* $\pi_1 : A \times B \to A$ and $\pi_2 : A \times B \to B$; for each object $C \in \mathrm{Ob}$ and a pair of morphisms $f : C \to A$ and $g : C \to B$, there's a **unique** *pairing morphism* $[f;g] : C \to A \times B$ such that the following properties hold:

$$\pi_1 \circ [f;g] = f \qquad\qquad \pi_2 \circ [f;g] = g$$

(5) For each $A, B \in \mathrm{Ob}$ there's their *exponential object* B^A together with *evaluation morphism* $\mathrm{ev} : B^A \times A \to B$; for each object $C \in \mathrm{Ob}$ and morphism $f : C \times A \to B$, there's a **unique** *abstraction* $\lambda f : C \to B^A$ such that the following property holds for arbitrary $g : C \to A$:

$$\mathrm{ev} \circ [\lambda f; g] = f \circ [\mathrm{id}_C; g]$$

Then, a well-typed term $\Gamma \vdash t : T$ is interpreted as a morphism $[\![t]\!] : [\![\Gamma]\!] \to [\![T]\!]$, where $[\![\Gamma]\!]$ is a left-nested product of interpretations of types from Γ:

$$[\![x : A, y : B, z : C]\!] = ((\top \times [\![A]\!]) \times [\![B]\!]) \times [\![C]\!].$$

Unit type is now a terminal object; product of types is now a product of objects; arrow type is an exponential object. Equations in (2) are needed to validate structural properties of substitution; equations in (4) correspond to reductions of pairs; equation in (5) validates β-reduction. **Unique**ness properties correspond to η-equivalence.

For example, in our language, $[\![t]\!]\,\mathcal{M}$ is an operator of reinterpretation of term t in internal model $\mathcal{M}$. On a sample term

$$\cdot \vdash \lambda f x y.(f x, y) : (A \to B) \to A \to C \to B \times C$$

it computes like this:

$$[\![\lambda f x y.(f x, y))]\!]\mathcal{M} =$$
$$= \mathtt{lam}\,(\mathtt{lam}\,(\mathtt{lam}\,(\mathtt{pair}\,(\mathtt{eval} \circ \mathtt{pair}\,(\mathtt{proj}_2 \circ \mathtt{proj}_1 \circ \mathtt{proj}_1)\,(\mathtt{proj}_2 \circ \mathtt{proj}_1))\,\mathtt{proj}_2)))$$

Where `lam`, `pair` and `eval` are correspondingly abstraction morphism, pairing morphism and evaluation morphism taken from $\mathcal{M}$, defined in the same way as in the definition 3.1, translated into types of our system.

Now, moving on to the actual design of our language. As discussed in section 2, for a non-dependent case we're now in, we're going to only interpret normal forms of closed terms. Among other things, it means that we actually don't need equations and **unique**nesses from definition 3.1 to hold in our internal models! This approach doesn't break any of the metatheoretical properties considered in section 3.1. Additionally, for the sake of example, we're only going to consider models where objects are types. Then, internalization of a definition 3.1 would require 1) type

MonoType	S, T, C, P, E, TT	$::=$	α	type variable
		$\mid$	$\top$	unit type
		$\mid$	$S \times T$	product type
		$\mid$	$S \to T$	arrow type
PolyType	σ	$::=$	T	monomorphic type
		$\mid$	$\forall \alpha.\sigma$	forall type
Term	s, t	$::=$	x	variable
		$\mid$	$()$	singleton
		$\mid$	(s, t)	pair
		$\mid$	$\pi_1\, t$	first projection
		$\mid$	$\pi_2\, t$	second projection
		$\mid$	$\lambda x.t$	abstraction
		$\mid$	$t\ s$	application
		$\mid$	$\mathtt{let}\, x = s\, \mathtt{in}\, t$	polymorphic binding
		$\mid$	$x^{\mathcal{M}}$	reinterpretation
Model	$\mathcal{M}$	$::=$	$\langle T, t, \ldots \rangle$	model def. (see typing rules)

Fig. 1. HM-R terms and values

operators to define types of morphisms, products and exponential objects; 2) polymorphic functions to operationalize the $\forall\exists$ introductions of constructions in a model. Actually, we're going to get rid of type operators as well, since we can make do with just types-in-a-context which are then substituted into definition of a model for checking term-level operations. This way, our language is a small extension to classic Hindley-Milner type system, hence the name HM-R; grammar rules are collected in a figure 1. You'll notice that, despite our description, grammar doesn't define a reinterpretation operator $[\![t]\!]\, \mathcal{M}$ for arbitrary term t; instead, only variables get to be reinterpreted. In this system, reinterpretation is nothing but a total function on closed values.

Definition 3.2 (Values of HM-R). Values are denoted by metavariables u, v in the BNF grammar below. Metavariable n denotes neutral terms:

$$u, v ::= n \mid () \mid (u, v) \mid \lambda x.v$$
$$n ::= x \mid \pi_i\, n \mid n\, v$$

Thus, in order to define reinterpretation, it is enough to consider grammar forms of STLC:

Definition 3.3 (Reinterpretation of terms in HM-R).

$$[\![()]\!]^{\mathcal{M}}/\gamma = \mathtt{cnst} \tag{1}$$
$$[\![(s, t)]\!]^{\mathcal{M}}/\gamma = \mathtt{pair}\, [\![s]\!]^{\mathcal{M}}/\gamma\, [\![t]\!]^{\mathcal{M}}/\gamma \tag{2}$$
$$[\![\pi_i\, t]\!]^{\mathcal{M}}/\gamma = \mathtt{comp}\, \mathtt{proj}_i\, [\![t]\!]^{\mathcal{M}}/\gamma \tag{3}$$
$$[\![x]\!]^{\mathcal{M}}/\gamma = \gamma\, x \tag{4}$$
$$[\![\lambda x.t]\!]^{\mathcal{M}}/\gamma = \mathtt{lam}\, [\![t]\!]^{\mathcal{M}}/[\vec{y} \coloneqq \mathtt{comp}\, \mathtt{proj}_1(\gamma\, \vec{y}),\ x \coloneqq \mathtt{proj}_2] \tag{5}$$
$$[\![t\, s]\!]^{\mathcal{M}}/\gamma = \mathtt{comp}\, \mathtt{eval}\, \left(\mathtt{pair}\, [\![t]\!]^{\mathcal{M}}/\gamma\, [\![s]\!]^{\mathcal{M}}/\gamma\right) \tag{6}$$

where model operations $\mathtt{cnst}, \mathtt{pair}$ and others are taken from $\mathcal{M}$ and γ is a substitution.

Here is our way to enforce a principle that *only closed terms get interpreted*: once a term s bound by a polymorphic let-binding $\mathtt{let}\, x = s\, \mathtt{in}\, t$ becomes closed and is brought into a normal form,

$$\frac{s \rightsquigarrow s'}{(s,t) \rightsquigarrow (s',t)} \qquad \frac{t \rightsquigarrow t'}{(s,t) \rightsquigarrow (s,t')} \qquad \frac{t \rightsquigarrow t'}{\pi_i t \rightsquigarrow \pi_i t'} \qquad \pi_i(t_1,t_2) \rightsquigarrow t_i$$

$$\frac{t \rightsquigarrow t'}{\lambda x.t \rightsquigarrow \lambda x.t'} \qquad \frac{s \rightsquigarrow s'}{s\,t \rightsquigarrow s'\,t} \qquad \frac{t \rightsquigarrow t'}{s\,t \rightsquigarrow s\,t'} \qquad (\lambda x.t)s \rightsquigarrow t[x \coloneqq s]$$

$$\frac{s \rightsquigarrow s'}{\mathtt{let}\,x = s\,\mathtt{in}\,t \rightsquigarrow \mathtt{let}\,x = s'\,\mathtt{in}\,t} \qquad \frac{t \rightsquigarrow t'}{\mathtt{let}\,x = s\,\mathtt{in}\,t \rightsquigarrow \mathtt{let}\,x = s\,\mathtt{in}\,t'}$$

$$\frac{\mathrm{fv}(v) = \varnothing}{\mathtt{let}\,x = v\,\mathtt{in}\,t \rightsquigarrow t[\![x \coloneqq v]\!]}$$

Fig. 2. HM-R computation rules

only then it is substituted into t, and whenever it is substituted into $x^{\mathcal{M}}$, it then immediately gets reinterpreted. This operation is denoted by $t[\![x \coloneqq s]\!]$ in our computation rules given in figure 2. Once again, it acts like a regular substitution in all cases except for variables under reinterpretation:

$$\left(x^{\mathcal{M}}\right)[\![x \coloneqq s]\!] = s^{\mathcal{M}}/\varepsilon \quad (\varepsilon \text{ is an empty substitution})$$

One can easily notice that computation would get stuck if let-bound terms were to contain variables coming from unapplied λ-abstractions. Our typing rules given in figure 3 protect us from that, allowing to refer only to let-bound terms inside other let-bound terms. Again, aside from that and typing rule for universes, there's nothing different from usual declarative typing rules for a Hindley-Milner type system [6].

To check models, we abuse our type system a bit in order to avoid adding type operators and complicate the type system: types of morphisms C, product objects P and exponentials E are specified as open monomorphic types which can use two type variables, 0 and 1. This fact is reflected in the syntax of a model using notation $\Lambda 01.C$, $\Lambda 01.P$ and $\Lambda 01.E$ respectively. 0 and 1 are then filled in by substitution when we need to type model operations (notation $T[S_1, S_2]$ is a shorthand for $T[0 \coloneqq S_1, 1 \coloneqq S_2]$). Terminal object is just a type named TT.

To check reinterpretation, we need to define how model acts on monomorphic types. Since there's no computation on them, reinterpretation of types is once again defined as a recursive function on syntax:

Definition 3.4 (Reinterpretation of types in HM-R). Let $\mathcal{M}$ be a correct model and σ be a substitution of monomorphic types. Then, interpretation of type in a model is the following function:

$$[\![T]\!]^{\mathcal{M}}_{\sigma} = C[TT, T^{\mathcal{M}}_{\sigma}] \tag{7}$$

$$\alpha^{\mathcal{M}}_{\sigma} = \sigma\alpha \tag{8}$$

$$\top^{\mathcal{M}}_{\sigma} = TT \tag{9}$$

$$(S \times T)^{\mathcal{M}}_{\sigma} = P[S^{\mathcal{M}}_{\sigma}, T^{\mathcal{M}}_{\sigma}] \tag{10}$$

$$(S \to T)^{\mathcal{M}}_{\sigma} = E[S^{\mathcal{M}}_{\sigma}, T^{\mathcal{M}}_{\sigma}]. \tag{11}$$

$$\Gamma \vdash () : \top \qquad \frac{\Gamma \vdash s : S \qquad \Gamma \vdash t : T}{\Gamma \vdash (s,t) : S \times T} \qquad \frac{\Gamma \vdash t : S \times T}{\Gamma \vdash \pi_1\, t : S} \qquad \frac{\Gamma \vdash t : S \times T}{\Gamma \vdash \pi_2\, t : T}$$

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \qquad \frac{\Gamma, x : S \vdash t : T}{\Gamma \vdash \lambda x.t : S \to T} \qquad \frac{\Gamma \vdash t : S \to T \qquad \Gamma \vdash s : S}{\Gamma \vdash t\, s : T}$$

$$\frac{\forall\Gamma \vdash s : S \qquad \mathrm{fv}(S) = \vec{\alpha} \qquad \forall\Gamma, x : \forall\vec{\alpha}.S, \Delta \vdash t : T}{\forall\Gamma, \Delta \vdash \mathtt{let}\, x = s\, \mathtt{in}\, t : T} \qquad \frac{x : \forall\vec{\alpha}.T \in \forall\Gamma \qquad \mathcal{M}\ \mathrm{model}}{\forall\Gamma, \Delta \vdash x^{\mathcal{M}} : [\![T]\!]^{\mathcal{M}}_{\left[\vec{\alpha} := \vec{S}\right]}}$$

$$\frac{\begin{array}{c}\mathrm{fv}(C) \cup \mathrm{fv}(P) \cup \mathrm{fv}(E) \subseteq \{0,1\} \qquad \Gamma \vdash \mathtt{id} : C[\alpha,\alpha] \qquad \Gamma \vdash \mathtt{comp} : C[\beta,\gamma] \to C[\alpha,\beta] \to C[\alpha,\gamma] \\ \mathrm{fv}(TT) = \varnothing \qquad \Gamma \vdash \mathtt{cnst} : C[\alpha, TT] \qquad \Gamma \vdash \mathtt{pair} : C[\gamma,\alpha] \to C[\gamma,\beta] \to C[\gamma, P[\alpha,\beta]] \\ \Gamma \vdash \mathtt{proj}_1 : C[P[\alpha,\beta],\alpha] \qquad \Gamma \vdash \mathtt{proj}_2 : C[P[\alpha,\beta],\beta] \\ \Gamma \vdash \mathtt{lam} : C[P[\gamma,\beta],\alpha] \to C[\gamma, E[\alpha,\beta]] \qquad \Gamma \vdash \mathtt{eval} : C[P[E[\alpha,\beta],\beta],\alpha]\end{array}}{\Gamma \vdash \langle \Lambda 01.C, \mathtt{id}, \mathtt{comp}, TT, \mathtt{cnst}, \Lambda 01.P, \mathtt{pair}, \mathtt{proj}_1, \mathtt{proj}_2, \Lambda 01.E, \mathtt{lam}, \mathtt{eval}\rangle\ \mathrm{model}}$$

Fig. 3. HM-R declarative typing rules

3.1 Metatheoretical properties of HM-R

Here we only study some basic properties of a system to make sure that everything works as expected: computations don't diverge; types are preserved by all computations; all of the types can be inferred if you wish to do so; everything terminates and, modulo reduction of `let`-bindings, this is the same old STLC we started with. Proofs are mostly following standard techniques, since rules for reinterpretation are structured to directly plug into proofs of properties of STLC and Hindley-Milner type system.

Theorem 3.5 (Church-Rosser theorem for HM-R). *Reflexive transitive closure of* $(\to)$ *has Church-Rosser property.*

Proof. Classic proof via parallel reduction relation [17]. Parallel reduction is defined as:

$$t \twoheadrightarrow t \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{(s,t) \twoheadrightarrow (s',t')} \qquad \frac{t \twoheadrightarrow t'}{\pi_i\, t \twoheadrightarrow \pi_i\, t'} \qquad \frac{t_1 \twoheadrightarrow t_1' \qquad t_2 \twoheadrightarrow t_2'}{\pi_i(t_1,t_2) \twoheadrightarrow t_i'}$$

$$\frac{t \twoheadrightarrow t'}{\lambda x.t \twoheadrightarrow \lambda x.t'} \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{s\, t \twoheadrightarrow s'\, t'} \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{(\lambda x.s)\, t \twoheadrightarrow s'[x \coloneqq t']}$$

$$\frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{\mathtt{let}\, x = s\, \mathtt{in}\, t \twoheadrightarrow \mathtt{let}\, x = s'\, \mathtt{in}\, t'} \qquad \frac{t \twoheadrightarrow t'}{\mathtt{let}\, x = v\, \mathtt{in}\, t \twoheadrightarrow t'[\![x \coloneqq v]\!]}$$

□

Theorem 3.6 (Type preservation for HM-R). *The following is true:*

(1) *If* $\mathcal{M}$ model, $\gamma\, x : C[\Gamma^{\mathcal{M}}, \Gamma(x)^{\mathcal{M}}]$ *for all* $x \in \Gamma$ *and* $\Gamma \vdash t : T$, *then* $[\![t]\!]^{\mathcal{M}}/\gamma : C[\Gamma^{\mathcal{M}}, T^{\mathcal{M}}]$.
(2) *If* $\Gamma \vdash s : S$ *and* $\Gamma, x : S, \Delta \vdash t : T$, *then* $\Gamma, \Delta \vdash t[x \coloneqq s] : T$.
(3) *If* $\Gamma \vdash v : \sigma$ *and* $\Gamma, x : \sigma, \Delta \vdash t : T$, *then* $\Gamma, \Delta \vdash t[\![x \coloneqq v]\!] : T$.
(4) *If* $\Gamma \vdash t : T$ *and* $t \rightsquigarrow t'$, *then* $\Gamma \vdash t' : T$.

Proof. By structural induction on terms and $(\rightsquigarrow)$. □

Theorem 3.7 (Strong normalization for HM-R). *Closed terms typable in HM-R are strongly normalizing with respect to* $(\rightsquigarrow)$ *and reduce to a value.*

Proof. By Tait's method of logical relations. Let SN be a set of strongly normalizing terms which reduce to a value. Let there be the following relation $\mathcal{R} \subseteq \text{PolyType} \times \text{Term}$:

- $t \in \mathcal{R}_\alpha, \mathcal{R}_\top$ iff $t \in SN$;
- $t \in \mathcal{R}_{T_1 \times T_2}$ iff $\pi_i\, t \in \mathcal{R}_{T_i}$ for $i = 1, 2$;
- $t \in \mathcal{R}_{S \to T}$ iff for all $s \in \mathcal{R}_S$, $t\, s \in \mathcal{R}_T$;
- $t \in \mathcal{R}_{\forall \vec{\alpha}.T}$ iff t is closed and $t \in \mathcal{R}_{T[\vec{\alpha} \coloneqq \vec{S}]}$ for all possible $\vec{S}$.

Note that $\mathcal{R}_T \subseteq SN$. Furthermore, the following property holds: if $t \in \mathcal{R}_{\forall \vec{\alpha}.T}$, then $[\![\mathrm{nf}(t)]\!]^{\mathcal{M}}/\varepsilon \in \mathcal{R}_{[\![T]\!]\,\mathcal{M}\,[\vec{\alpha} \coloneqq \vec{S}]}$ for all $\mathcal{M}$ s.t. all terms inside it are in corresponding $\mathcal{R}_\forall$.

Then we can prove our main lemma: if $\Gamma \vdash t : T$ and γ is a substitution s.t. $\gamma\, x \in \mathcal{R}_{\Gamma(x)}$ for all $x \in \Gamma$, then $\gamma\, t \in \mathcal{R}_T$. □

Theorem 3.8 (Conservativity over STLC). *Normal forms of closed terms typable in HM-R are typable in STLC.*

Proof. Thanks to type preservation, normal form $\mathrm{nf}(t)$ is typable in HM-R. By definition, it is a value, therefore it has no reinterpretation subterms and let-bindings, hence typable in STLC. □

Theorem 3.9. *Global type inference for HM-R is decidable.*

Proof. It is enough to extend any unification-based type inference algorithm for Hindley-Milner type system to support models and reinterpretation.

- Since models are provided explicitly, it is enough to *check* them.
- Since polymorphic functions do not depend on local context, their type is fully resolved when it's added to context.
- Once model is checked in $x^{\mathcal{M}}$, we can directly compute $[\![T]\!]^{\mathcal{M}}_{[\vec{\alpha} \coloneqq \vec{\eta}]}$ (since $\mathcal{M}$ is known and T is fully resolved) where $\vec{\eta}$ are fresh metavariables.

□

Thus, in this section we introduced a sound type system based on Hindley-Milner type system which is capable of interpreting its own closed terms in arbitrary "cartesian closed category" definable inside it. This system is not truly *reflective* since models are not fully internal: they require special form of type operators to work which is not accessible to the rest of type system. To internalize model definition, we move on to dependently typed system in the next section.

4 Almost reflective dependently typed language: MLTT-R

Now we are ready to take on the main challenge: design a minimal dependently typed reflective language, and provide internal models of itself to yield universe polymorphism, external parametricity and do synthetic mathematics inside it. As a starting point, we take Martin-Löf type theory with $\Pi-, \Sigma-$, Id− types, natural number type and infinite non-cumulative hierarchy of universes. As discussed in section 2, the best practical solution to reinterpretation here would be to forbid reductions under it and reinterpret terms as-is, and constrain models using Id-types. Still, in comparison to non-dependently typed case, there are infinitely more problems to solve if we want to define a truly reflective system:

(1) If we naïvely adopt an approach to typing of models from section 3, in order to specify a model, we would need to specify countably many models, one per each universe. And even if we did, to which universe does the type of models belong?
(2) Now that we have computations in types, we have to deal with conversion rule under reinterpretation:

$$\frac{t : T \quad T \equiv T'}{t : T'} \overset{?}{\rightsquigarrow} \frac{[\![t]\!] : [\![T]\!] \quad [\![T]\!] \equiv [\![T']\!]}{[\![t]\!] : [\![T']\!]}$$

The problem is that, after interpretation, $[\![T]\!]$ is only *propositionally* equal to $[\![T']\!]$.
(3) If we want a truly reflective system, we need to know what to do with *reinterpretation of reinterpretation*:

$$\lambda x.[\![[\![x]\!]_1]\!]_2 \quad \text{and} \quad [\![\lambda x.[\![x]\!]_1]\!]_2$$

In the first case, we perform two subsequent interpretations on a yet unknown x; in the second case, we interpret the process of interpretation itself. What should this mean?

To deal with universe sizing issues, consider the following observation: in a system without level binders, each term uses only a finite number of levels. Hence to reinterpret this term, we need to only provide a finite number of universe models, which can together fit into some larger universe of a finite level. No universe polymorphism required!

To interpret conversion rule, we start by first pinpointing the locations where conversion rule is actually used; in order to do this, we *need* to use bidirectional type checking. At the moment of conversion, interpretation has enough information to know both T and T'; thus we can translate a chain of reductions performed during definitional equality check into a chain of propositional equalities taken from coherence conditions proven in the definition of a model. Then, a point of conversion is wrapped in equality elimination with synthesized witness as a scrutinee.

In reinterpreting reinterpretation, we actually only deal with "double reinterpretation": cases like $\lambda x.[\![[\![x]\!]_1]\!]_2$ can be dealt with by *merging* two models into a new one, a technique reminiscent of *Collapsing towers of interpreters* (TODO: cite). We do not try to actually interpret interpretation itself as in $[\![\lambda x.[\![x]\!]_1]\!]_2$ and introduce a modality $\Box$ meaning "no interpretation was used in construction of this term"; reinterpretable terms have to have a boxed type. We also use $\Box$ to relax a ban on computations inside reinterpretation by allowing "splicing" boxed expressions into other boxed expressions; inside splices, computation is allowed to bring terms to the boxed form for further interpretation.

Note that discussion above was ambient to the choice of formalism to present models. In line with current state of the art, we use Categories-with-Families [5] as one of the most convenient frameworks used today to specify models of dependent type theories.

TODO: CwF definition

To actually compute interpretation in MLTT-R, we need two mutually recursive functions: $t \mathbin{@}^{\mathcal{Y}}_{\Leftarrow T} \mathcal{U}$ and $t \mathbin{@}^{\mathcal{Y}}_{\Rightarrow} \mathcal{U}$ for checkable and synthesizing terms, respectively. TODO: FINISH.

4.1 Metatheoretical properties of MLTT-R

Once again, we only try to make sure of the most essential properties: that computations do not diverge, that types during computations are preserved, all computations terminate and that resulting type system is a conservative extension of MLTT.

Theorem 4.1 (Church-Rosser theorem for STLC-R). *Reflexive transitive closure of* $(\rightarrow)$ *has Church-Rosser property.*

Proof. Classic proof via parallel reduction relation [17]. □

Context	Γ	::=	ε	empty global context
		|	$\Gamma;\Theta$	localization
Small context	Θ	::=	$\cdot$	empty small context
		|	$\Theta, x : T$	variable extension
Type	T, S	::=	$\mathcal{U}_\ell$	universe
		|	$\mathrm{El}_\ell(t)$	universal type family
		|	Nat_ℓ	natural number type
		|	$(x : S) \times T$	Σ-type
		|	$(x : S) \to T$	Π-type
		|	$t_1 =_T t_2$	identity type
		|	$\Box\, T$	type of closed terms
		|	$\mathcal{M}_{\ell_0,\ldots,\ell_\kappa}$	type of models
		|	T^t_Γ	typal reinterpretation
Level	ℓ, κ, j	$\in$	$\mathbb{N}$	finite levels
Term	t, s	::=	$x \mid y \mid z \mid \mathtt{m} \mid \ldots$	variables
		|	U_ℓ	code of a universe
		|	$\mathtt{Nat}_\ell$	code of a natural number type
		|	0	zero
		|	$\mathsf{S}\, t$	successor
		|	$\mathrm{natind}_{x.T}\, t\, \{s_0, y \mapsto s_S\}$	induction
		|	$\mathtt{Sigma}(x : s)t$	code of a Σ-type
		|	(s, t)	pairing
		|	$\pi_1\, t \mid \pi_2\, t$	projections
		|	$\mathtt{Pi}(x : s)t$	code of a Π-type
		|	$\lambda x.\, t$	abstraction
		|	$t\, s$	application
		|	$\mathtt{Id}\, t\, t_1\, t_2$	code of an identity type
		|	refl	reflexivity
		|	$\mathrm{J}_{xy.S}\, t\, s$	J rule (elimination of equality type)
		|	$\mathtt{Box}\, t$	code of a box type
		|	$[t]_{🔒}$	locking
		|	$[t]_{🔓}$	unlocking
		|	$[\![t]\!]^{\ell_0,\ldots,\ell_\kappa}$	internal interpretation
		|	t^s_Γ	intermediate interpretation
		|	$t : T$	ascription

Fig. 4. MLTT-R grammar

Theorem 4.2 (Type preservation for MLTT-R).

Proof. By structural induction on terms and $(\to)$. □

Theorem 4.3 (Strong normalization for MLTT-R). *Closed terms typable in MLTT-R are strongly normalizing with respect to* $(\to)$.

Proof. Tait's method of logical relations for MLTT with modifications similar to ones performed in a proof of strong normalization for STLC-R should suffice. However, we're not completely sure. □

$$\textsc{WF-Loc}\ \frac{\Gamma \vdash \Theta}{\Gamma; \Theta\ \mathrm{wf}} \qquad \textsc{WF-Eps}\ \varepsilon \vdash \cdot \qquad \textsc{WF-Nil}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \cdot} \qquad \textsc{WF-Var}\ \frac{\Gamma \vdash \Theta \qquad \Gamma; \Theta \vdash \mathrm{level}(T) = \ell}{\Gamma \vdash \Theta, x : T}$$

$$\textsc{Level-Uni}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathrm{level}(\mathcal{U}_\ell) = \ell + 1} \qquad \textsc{Level-El}\ \frac{\Gamma \vdash t \Leftarrow \mathcal{U}_\ell}{\Gamma \vdash \mathrm{level}(\mathrm{El}_\ell(t)) = \ell} \qquad \textsc{Level-Nat}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathrm{level}(\mathrm{Nat}_\ell) = \ell} \qquad \textsc{Level-Box}\ \frac{\Gamma; \cdot \vdash \mathrm{level}(T) = \ell}{\Gamma \vdash \mathrm{level}(\Box\, T) = \ell}$$

$$\textsc{Level-Sigma, Level-Pi}\ \frac{\Gamma; \Theta \vdash \mathrm{level}(S) = \kappa \qquad \Gamma; \Theta, x : S \vdash \mathrm{level}(T) = \ell}{\Gamma; \Theta \vdash \mathrm{level}((x : S) \times T) = \mathrm{level}((x : S) \to T) = \max(\kappa, \ell)}$$

$$\textsc{Level-Id}\ \frac{\Gamma \vdash \mathrm{level}(T) = \ell \qquad \Gamma \vdash t_1, t_2 \Leftarrow T}{\Gamma \vdash \mathrm{level}(t_1 =_T t_2) = \ell} \qquad \textsc{Level-Model}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathrm{level}\left(\mathcal{M}_{\ell_0,\ldots,\ell_\kappa}\right) = \max(\ell_0, \ldots, \ell_\kappa) + 1}$$

$$\textsc{Level-TyInterp}\ \frac{\Gamma \vdash t \Rightarrow \mathcal{M}_{\ell_0,\ldots,\ell_\kappa} \qquad \Gamma \vdash \Theta \qquad \Gamma; \Theta \vdash \mathrm{level}(T) = j \qquad j \leqslant \kappa}{\Gamma \vdash \mathrm{level}\left(T^t_\Delta\right) = \ell_j}$$

Fig. 5. Context well-formedness & level placement rules

Theorem 4.4 (Conservativity over MLTT). *For MLTT context* Γ*, MLTT type* T *and MLTT-R term* t*, if* $\Gamma \vdash t \Leftarrow T$*, then* $\Gamma \vdash_{\mathrm{MLTT}} \mathrm{nf}(t) : T$.

Proof. Thanks to type preservation, normal form $\mathrm{nf}(t)$ is typable in MLTT-R. TODO TODO TODO. Therefore $\mathrm{nf}(t)$ is an MLTT term and is typable in MLTT. □

4.2 Useful reinterpretations inside MLTT-R

Here we list some of the language features which reinterpretation enables without any additional language support.

4.2.1 Crude, but effective stratification. Due to McBride [13, 14], a technique of crude, but effective stratification works as follows: instead of providing complete support for level polymorphism, let's consider a *displacement operator* which, given a term t using universes $U_i, U_j, U_k, \ldots$, produces a term which is equal to t except for universes which are replaced with $U_{i+1}, U_{j+1}, U_{k+1}, \ldots$. The displaced term is also well-typed, however it is defined «one level higher». Notice that displacement operator is *definable* in MLTT-R as $_ @ \mathcal{U}_{\mathrm{McBride}}$ where

$$\mathcal{U}_{\mathrm{McBride}} = \langle \Lambda l. U_{suc\, l}, \ldots \rangle$$

4.2.2 Displayed type theory.

4.2.3 Internal external parametricity. Demonstrated by Reynolds [16], parametricity is a property of a type system that, in simple words, «polymorphic function cannot meaningfully depend on generic inputs». Thus any polymorphic function of type σ satisfies a certain «free theorem» $[\![\sigma]\!]$ defined via relational interpretation of a type.

$$\textsc{Infer-Uni}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathsf{U}_\ell \Rightarrow \mathcal{U}_{\ell+1}} \qquad \textsc{Infer-Nat}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathsf{Nat}_\ell \Rightarrow \mathcal{U}_\ell} \qquad \textsc{Check-Zero}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash 0 \Leftarrow \mathrm{Nat}_\ell} \qquad \textsc{Check-Succ}\ \frac{\Gamma \vdash t \Leftarrow \mathrm{Nat}_\ell}{\Gamma \vdash \mathsf{S}\,t \Leftarrow \mathrm{Nat}_\ell}$$

$$\textsc{Infer-NatInd}\ \frac{\Gamma \vdash t \Rightarrow \mathrm{Nat}_\ell \qquad \Gamma, x : \mathrm{Nat}_\ell \vdash \mathrm{level}(T) = _ \qquad \Gamma \vdash s_0 \Leftarrow T[x \coloneqq 0] \qquad \Gamma, y : \mathrm{Nat}_\ell, z : T[x \coloneqq y] \vdash s_S \Leftarrow T[x \coloneqq \mathsf{S}\,y]}{\Gamma \vdash \mathrm{natind}_{x.T}\ t\ \{s_0, yz.s_S\} \Rightarrow T[x \coloneqq t]}$$

$$\textsc{Infer-Sigma}\ \frac{\Gamma \vdash s \Rightarrow \mathcal{U}_\kappa \qquad \Gamma, x : \mathrm{El}_\kappa(s) \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash \mathsf{Sigma}(x : s)t \Rightarrow \mathcal{U}_{\max(\kappa,\ell)}} \qquad \textsc{Check-Pair}\ \frac{\Gamma \vdash s \Leftarrow S \qquad \Gamma \vdash t \Leftarrow T[x \coloneqq s]}{\Gamma \vdash (s, t) \Leftarrow (x : S) \times T}$$

$$\textsc{Infer-Proj}_1\ \frac{\Gamma \vdash t \Rightarrow (x : S) \times T}{\Gamma \vdash \pi_1\,t \Rightarrow S} \qquad \textsc{Infer-Proj}_2\ \frac{\Gamma \vdash t \Rightarrow (x : S) \times T}{\Gamma \vdash \pi_2\,t \Rightarrow T[x \coloneqq \pi_1\,t]} \qquad \textsc{Infer-Pi}\ \frac{\Gamma \vdash s \Rightarrow \mathcal{U}_\kappa \qquad \Gamma, x : \mathrm{El}_\kappa(s) \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash \mathsf{Pi}(x : s)t \Rightarrow \mathcal{U}_{\max(\kappa,\ell)}}$$

$$\textsc{Check-Abs}\ \frac{\Gamma, x : S \vdash t \Leftarrow T}{\Gamma \vdash \lambda x.\,t \Leftarrow (x : S) \to T} \qquad \textsc{Infer-App}\ \frac{\Gamma \vdash t \Rightarrow (x : S) \to T \qquad \Gamma \vdash s \Leftarrow S}{\Gamma \vdash t\,s \Rightarrow T[x \coloneqq s]}$$

$$\textsc{Infer-Id}\ \frac{\Gamma \vdash t \Rightarrow \mathcal{U}_\ell \qquad \Gamma \vdash t_1, t_2 \Leftarrow \mathrm{El}_\ell(t)}{\Gamma \vdash \mathsf{Id}\,t\,t_1\,t_2 \Rightarrow \mathcal{U}_\ell} \qquad \textsc{Check-Refl}\ \frac{\Gamma \vdash t_1 \equiv t_2 : T}{\Gamma \vdash \mathrm{refl} \Leftarrow t_1 =_T t_2}$$

$$\textsc{Infer-J}\ \frac{\Gamma \vdash t \Rightarrow t_1 =_T t_2 \qquad \Gamma, x : T, y : t_1 =_T x \vdash \mathrm{level}(S) = _ \qquad \Gamma \vdash s \Leftarrow S[x \coloneqq t_1, y \coloneqq \mathrm{refl}]}{\Gamma \vdash \mathrm{J}_{xy.S}\,t\,s \Rightarrow S[x \coloneqq t_2, y \coloneqq t]}$$

$$\textsc{Infer-Var}\ \frac{🔒 \notin \Delta}{\Gamma, x : T, \Delta \vdash x \Rightarrow T} \qquad \textsc{Infer-Box}\ \frac{\Gamma.🔒 \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash \mathsf{Box}\,t \Rightarrow \mathcal{U}_\ell} \qquad \textsc{Check-Lock}\ \frac{\Gamma.🔒 \vdash t \Leftarrow T}{\Gamma \vdash [t]_🔒 \Leftarrow \Box T} \qquad \textsc{Infer-Unlock}\ \frac{\Gamma^{🔓} \vdash t \Rightarrow \Box T}{\Gamma \vdash [t]_{🔓} \Rightarrow T}$$

$$\textsc{Infer-Interp}\ \frac{\Gamma \vdash t \Rightarrow \Box T \qquad \Gamma.🔒 \vdash \mathrm{level}(T) = \kappa}{\Gamma \vdash [\![t]\!]^{\ell_0,\ldots,\ell_\kappa} \Rightarrow \left(\mathsf{m} : \mathcal{M}_{\ell_0,\ldots,\ell_\kappa}\right) \to T^{\mathsf{m}}_{\cdot}} \qquad \textsc{Infer-Interm-Interp}\ \frac{\Gamma \vdash s \Rightarrow \mathcal{M}_{\ell_0,\ldots,\ell_\kappa} \qquad \Gamma.🔒, \Delta \vdash t \Rightarrow T}{\Gamma \vdash t^s_\Delta \Rightarrow T^s_\Delta}$$

$$\textsc{Infer-Ascr}\ \frac{\Gamma \vdash \mathrm{level}(T) = _ \qquad \Gamma \vdash t \Leftarrow T}{\Gamma \vdash t : T \Rightarrow T} \qquad \textsc{Check-Inferred}\ \frac{\Gamma \vdash t \Rightarrow S \qquad \Gamma \vdash S \equiv T}{\Gamma \vdash t \Leftarrow T}$$

Fig. 6. Bidirectional term typing rules

In recent works [1, 3], a streamlined definition is given for parametricity interpretation of dependently-typed systems dubbed «external parametricity». Once again, this relational interpretation is definable in our system *internally* with the help of a relational model $\mathcal{U}_{\mathrm{para}}$. Since interpreted

functions do not depend on local context, we sidestep issues with relational context highlighted by Altenkirch and others [1], thus result of the interpretation is a proper proof of a free theorem.

4.2.4 Synthetic mathematics. As thoroughly discussed in a recent paper by Nawrocki and others [15], thanks to MLTT being an internal language of certain $(\infty, 1)$-categories, we can do *synthetic mathematics* inside it: prove a theorem inside MLTT about types; interpret it in a model of MLTT and yield a proof about objects in that model. It is easy to see that reinterpretation operator we have in MLTT-R is sufficient to do synthetic mathematics in it and immediately apply results interpreted in a suitable model.

5 Discussion & related work

To the best of our knowledge, we didn't find any research which explores the topic at hand, but the idea is certainly in the air, as illustrated by the following brilliant papers.

Original work by Lambek [12] sets the tone of the future research, establishing correspondence between type systems based on lambda calculus with various kinds of categories, starting with correspondence between simply typed lambda calculus and cartesian closed categories.

Conal Elliott's "Compiling to Categories" [7] shows how to interpret (some subset of) Haskell in arbitrary `Category` (again, definable in Haskell) via GHC compiler plugin. That paper is a direct inspiration for this one. There is an important difference, though: reinterpretation is internal to our type systems, unlocking new patterns in the usage of reinterpretation, including (but not limited to) universe polymorphism, external parametricity and more. However, Conal's approach is more flexible since his translation to categories requires desired category to only implement primitives actually used for translation of a particular term at hand.

In a recent paper "A Certifying Proof Assistant for Synthetic Mathematics in Lean", Wojciech Nawrocki and others [15] adapt Conal's approach to Lean's system with dependent types, using Categories-with-Families to represent internal models and employing Lean's powerful metaprogramming facilities instead of a compiler plugin. In doing so, they are able to provide a framework for doing synthetic mathematics in Lean, extracting proofs done in MLTT applied to more specific categories. Thus our results in section 4.2.4 are close to the results shown in their paper. To add to that, their work is already implemented in practical proof assistant. Yet, our type system involves no metaprogramming since reinterpretation is internal to it, and it handles conversion rule gracefully. In addition, the focus is a bit different, where we use reinterpretation to express other polymorphic features of a language.

Conor McBride's "Crude but effective stratification" [13, 14] (systemic treatment of this idea is given by Favonia and others [9]) introduces an eye-opening idea of using *displacement* as a lightweight form of universe polymorphism: to reuse term typed at a lower universe level, lift all levels in a term by adding a constant shift s. From the point of view advocated by this paper, you can see it as an act of reinterpreting an expression in an internal model of type system where universe at level i is interpreted as a universe at level $i + s$ (section 4.2.1).

In "Parametricity and Dependent Types" by Jean-Philippe Bernardy, Patrik Jansson and Ross Paterson [3], authors introduce a direct translation procedure to yield parametricity theorems and proofs from polymorphic definitions in a dependently typed system. The definition of «external parametricity» we follow, though, is given in "Internal Parametricity, without an Interval" by Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi and Michael Shulman [1]. Once again, we can reformulate the translation they give as reinterpretation in an internal model. Altenkirch and others noted an issue with this approach: if we apply the external translation to the judgment with nonempty context, context becomes parametric, too; that's the reason they add `rel` operation

and yield type system with *computational properties of higher observational type theory*. We sidestep the issue with context since reinterpretation is applied only to terms whose context is also «reinterpretable» and is not affected by the translation (section 4.2.3).

5.1 What we could've done differently

5.1.1 Modality of global elements. One could hope to salvage a situation by providing a "modality of global elements" which is a convenient relaxation of a notion of closed terms.

Definition 5.1. A dependently-typed system with a type of functions ($\to$) has a *global elements modality* $\square$ if there is a one-hole term $\mathrm{nec}\ _$ and an atomic two-hole term $\mathrm{K}\ _\ _$ such that the following rules are admissible:

$$\frac{\cdot \vdash T\ \mathrm{type}}{\Gamma \vdash \square T\ \mathrm{type}} \qquad \frac{\cdot \vdash t : T}{\Gamma \vdash \mathrm{nec}\ t : \square T} \qquad \frac{\Gamma \vdash f : \square\,(S \to T) \qquad \Gamma \vdash t : \square S}{\Gamma \vdash K\ f\ t : \square T}$$

And, in addition, the following reduction is admissible: $K\ (\mathrm{nec}\ \lambda x.t)\ (\mathrm{nec}\ s) \to^* \mathrm{nec}(t[x \coloneqq s])$.

The intuition behind the reduction is that, if we apply a "global function" to a "global argument", we then should be able to "inline" this application, what this reduction precisely does.

Then, a corresponding weakening of computational reinterpretation can be defined as follows.

Definition 5.2 (Modal computational reinterpretation). We call a type operator $F[_]$ a modal computational reinterpretation of types if it is defined on boxed types (if $\Gamma \vdash \square T\ \mathrm{Type}$, then $\Gamma \vdash F[T]\ \mathrm{Type}$), respects computation on types (if $T \to T'$, then $F[T] \to^* F[T']$) and computes as a primitive recursive function on normal forms of types. Analogously, term with a hole $[\![_]\!]$ is a modal computational reinterpretation of terms if it respects computation on terms, computes as a primitive recursive function on normal forms of terms and, if $\Gamma \vdash t : \square T$, then $\Gamma \vdash [\![t]\!] : F[T]$.

Maybe, this definition of reintrerpretation is "nice"? Regrettably, there's a no-go theorem which is similar to naïve version 2.2 which holds in this case:

Theorem 5.3 (No-go theorem of modal reinterpretation). *In a confluent type system with modal computational reinterpretation, the following should hold for arbitrary well-typed s and t:*

$$\mathsf{app}[\![\mathrm{nec}\ \lambda x.t]\!][\![\mathrm{nec}\ s]\!] \equiv [\![\mathrm{nec}(t[x \coloneqq s])]\!]$$

where $\equiv$ is definitional equality and app *is how $[\![_]\!]$ acts on K.*

Proof. By direct example. We are given well-formed types $\cdot \vdash S, T\ \mathrm{type}$ and well-typed terms $\cdot \vdash s : S$ and $\cdot, x : S \vdash t : T$. Consider the following term:

$$\Gamma, x : \square(S \to T), y : \square\, S \vdash [\![K\ x\ y]\!] : F[T]$$

Since K is atomic, term under interpretation is in normal form, hence the whole term computes to $\mathsf{app}[\![x]\!][\![y]\!]$. Substituting $\mathrm{nec}\ \lambda x.t$ for x and $\mathrm{nec}\ s$ for y, we get $\mathsf{app}[\![\mathrm{nec}\ \lambda x.t]\!][\![\mathrm{nec}\ s]\!]$.

On the other hand, if we perform substitution first, we would get $\Gamma \vdash [\![K\ (\mathrm{nec}\ \lambda x.t)\ (\mathrm{nec}\ s)]\!] : F[T]$. Since modal reinterpretation respects reduction, this reduces to $[\![\mathrm{nec}(t[x \coloneqq s])]\!]$.

In a confluent type system with functions, multi-step reduction commutes with substitution, therefore two terms are definitionally equivalent. □

6 Conclusions and further work

In retrospect, we managed to define two self-interpreting type systems, STLC-R and MLTT-R, meaning that they have a notion of their own «internal models» which they then can use to reinterpret their own terms inside such models. In doing so, such type systems get new features which

previously were defined as separate builtin quirks of a system. Basic metatheoretical properties are proven; for future work we have the following plans:

- Provide mechanized proofs of metatheoretical properties of systems;
- Provide implementation of MLTT-R in a practical proof assistant;
- Handle «partial» interpretation a-la Conal Elliott's "Compiling to Categories" [7];
- Make equality in MLTT-R more extensional, most probably by making type system closer to Homotopy type theory [18];
- Relax context constraints in reinterpretable terms.

References

[1] Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi, and Michael Shulman. 2024. Internal Parametricity, without an Interval. *Proc. ACM Program. Lang.* 8, POPL, Article 78 (Jan. 2024), 30 pages. doi:10.1145/3632920

[2] Thorsten Altenkirch and Ondrej Rypacek. 2012. A Syntactical Approach to Weak omega-Groupoids. In *Computer Science Logic (CSL'12) - 26th International Workshop/21st Annual Conference of the EACSL (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 16)*, Patrick Cégielski and Arnaud Durand (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 16–30. doi:10.4230/LIPIcs.CSL.2012.16

[3] Jean-Philippe Bernardy, Patrik Jansson, and Ross Paterson. 2010. Parametricity and dependent types. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) *(ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 345–356. doi:10.1145/1863543.1863592

[4] Marc Bezem, Thierry Coquand, Peter Dybjer, and Martín Escardó. 2021. A Note on Generalized Algebraic Theories and Categories with Families. arXiv:2012.08370 [math.CT] https://arxiv.org/abs/2012.08370

[5] Simon Castellan, Pierre Clairambault, and Peter Dybjer. 2020. Categories with Families: Unityped, Simply Typed, and Dependently Typed. arXiv:1904.00827 [cs.LO] https://arxiv.org/abs/1904.00827

[6] Dominique Clément, Thierry Despeyroux, Gilles Kahn, and Joëlle Despeyroux. 1986. A simple applicative language: mini-ML. In *Proceedings of the 1986 ACM Conference on LISP and Functional Programming* (Cambridge, Massachusetts, USA) *(LFP '86)*. Association for Computing Machinery, New York, NY, USA, 13–27. doi:10.1145/319838.319847

[7] Conal Elliott. 2017. Compiling to categories. *Proc. ACM Program. Lang.* 1, ICFP, Article 27 (Aug. 2017), 27 pages. doi:10.1145/3110271

[8] Martin Hofmann and Thomas Streicher. 1994. The groupoid model refutes uniqueness of identity proofs. In *Proceedings of the Ninth Annual IEEE Symp. on Logic in Computer Science, LICS 1994* (Paris, France), Samson Abramsky (Ed.). IEEE Computer Society Press, 208–212.

[9] Kuen-Bang Hou (Favonia), Carlo Angiuli, and Reed Mullanix. 2023. An Order-Theoretic Analysis of Universe Polymorphism. *Proc. ACM Program. Lang.* 7, POPL, Article 57 (Jan. 2023), 27 pages. doi:10.1145/3571250

[10] W. Howard. 1980. The Formulae-as-Types Notion of Construction. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, J. Seldin and J. Hindley (Eds.). Academic Press, 479–490.

[11] Astra Kolomatskaia and Michael Shulman. 2025. Displayed type theory and semi-simplicial types. *Mathematical Structures in Computer Science* 35 (2025), e34. doi:10.1017/S096012952510025X

[12] Joachim Lambek. 1980. From λ-calculus to cartesian closed categories. In *To H. B. Curry: Essays on Combinatory Logic, Lambda-Calculus and Formalism*, J. P. Seldin and J. R. Hindley (Eds.). Academic Press, London, 375–402.

[13] Conor McBride. 2011. Crude but Effective Stratification. https://mazzo.li/epilogue/index.html%3Fp=857&cpage=1.html

[14] Conor McBride. 2012. Crude but Effective Stratification. https://personal.cis.strath.ac.uk/conor.mcbride/Crude.pdf

[15] Wojciech Nawrocki, Joseph Hua, Mario Carneiro, Yiming Xu, Spencer Woolfson, Shuge Rong, Sina Hazratpour, and Steve Awodey. 2026. A Certifying Proof Assistant for Synthetic Mathematics in Lean. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Rennes, France) *(CPP '26)*. Association for Computing Machinery, New York, NY, USA, 88–103. doi:10.1145/3779031.3779087

[16] John C Reynolds. 1983. Types, abstraction and parametric polymorphism. In *Information Processing 83*, R. E. A. Mason (Ed.). North-Holland, 513–523.

[17] Gregor Richards. 2025. Proof of the Church-Rosser Theorem. https://student.cs.uwaterloo.ca/~cs442/W25/extras/c-r-thm-proof.pdf

[18] The Univalent Foundations Program. 2013. *Homotopy Type Theory: Univalent Foundations of Mathematics*. https://homotopytypetheory.org/book, Institute for Advanced Study.